Smart Zebra: A Prompt-to-GRPO Recipe for Logic-Grid Reasoning in Small Language Models

Anonymous ACL submission

Abstract

Improving logical reasoning in small language models remains an open challenge. We introduce Smart Zebra, a full training recipe for Zebra puzzle solving, a demanding constraint satisfaction task where current models struggle substantially. Our pipeline begins with a publicly released reimplementation of the ZebraLogic puzzle generator, filling a reproducibility gap in the benchmark's original release. Training Qwen2.5-3B-Instruct under two prompt strategies — the one given by ZebraLogic baseline and a natural language adaptation of the AC-3 arc-consistency algorithm — we find AC-3 yields stronger in-distribution performance under SFT, while the baseline prompt improves further under GRPO post-training, with AC-3 degrading considerably. Even with straightforward supervised fine-tuning, we observe substantial gains in puzzle accuracy & cell accuracy over the few-shot baseline. Our generator, training code, and results are publicly released [1].

1 Introduction

Zebra puzzles — also known as Einstein's puzzles or logic grid puzzles — are a type of constraint satisfaction problem (CSP) that require matching M categorical attributes to N houses in accordance with a number of textual clues. Being NP-complete and highly structured with a single solution, they provide a controlled, verifiable testbed for evaluating logical reasoning in language models. Compared to other CSPs such as Sudoku, the linguistic nature of Zebra puzzles provides a more natural fit for testing language models.

Despite offering a rich testbed, Zebra puzzles remain underutilized in the NLP literature. The most prominent benchmark is ZebraLogic (Lin et al., 2025), which comprises 1,000 puzzles categorized by size (S, M, L, XL). Their results demonstrate that current LLMs struggle substantially with Zebra puzzle solving; however, evaluation was limited to few-shot prompting with identical prompts across all models tested. We propose a training recipe that first applies supervised fine-tuning (SFT) and then Group Relative Policy Optimization (GRPO) (Shao et al., 2024) with cell-accuracy rewards to improve puzzle-solving performance.

Tyagi et al. (2024) develop GridPuzzle, an evaluation dataset of 274 grid-based puzzles with different complexities, and test the reasoning performance of various LLMs on it. The study creates an error taxonomy for LLM-generated reasoning traces and provides an auto-evaluator.

Dziri et al. (2023) probe the limits of transformer LLMs in compositional tasks, including logic grid puzzles. Their results show that zero-shot performance collapses with increasing task complexity and declines sharply on out-of-domain examples.

Related work largely focuses on neurosymbolic approaches (Beiser et al., 2025; Amonkar et al., 2026), where the model translates a puzzle into a formal solver such as Z3 rather than solving it directly, or trains on Zebra puzzles primarily to improve general reasoning rather than puzzle-solving accuracy. A recent concurrent effort (Gupta and Gupta, 2025) attempts reinforcement learning steering using canonical solver-order trajectories; in contrast, we reward cell accuracy directly, imposing no rigid solver structure on the model.

The ZebraLogic benchmark provides 1,000 evaluation puzzles and a prose description of its generation pipeline but omits the puzzle generation source code itself. Because generating in-distribution training data was a prerequisite for training, we performed a complete reimplementation. To address this gap, we reimplement and release the ZebraLogic generator to establish a fully reproducible data pipeline and to enable the generation of additional puzzles. Also we utilize this pipeline to

[1] Code available at: https://anonymous.4open.science/r/Smart-Zebra-256

train Qwen2.5-3B-Instruct, a model also evaluated in the ZebraLogic benchmark.

We adopt ZebraLogic as our primary benchmark as their zebra puzzles provides unambiguous, automatically verifiable ground truth across four size categories. Finally, being the most widely used Zebra puzzle benchmark, it allows direct comparison with prior work.

Closest in spirit to our work, Chen et al. (2025) introduce Enigmata, a suite of 36 puzzle tasks each equipped with a controllable generator and a rule-based verifier, and train models via multi-task RLVR, demonstrating strong generalization on out-of-domain benchmarks. We use Enigmata-Eval as a secondary benchmark to probe whether in distribution gains reflect genuine reasoning improvements or format specific overfitting. Wong et al. (2025) similarly propose a "play to learn" framework that fine-tunes LLMs on seven custom logic puzzles via RL with verifiable rewards, showing significant improvements on mathematical reasoning benchmarks; however, their focus is on general reasoning transfer rather than puzzle-solving accuracy itself.

The main contributions of this work are:

- A fully reproducible reimplementation and open-source release of the ZebraLogic puzzle generator.
- Adaptation of the AC-3 algorithm (Mackworth, 1977) which is commonly used to solve constraint satisfaction problems and comparing the results with baseline throughout our pipeline.
- A staged training recipe combining SFT and GRPO (Shao et al., 2024) with cell-accuracy rewards, demonstrating measurable improvements over just the few-shot prompted Qwen2.5-3B-Instruct.

2 Method

All training and evaluation stages are conducted using Qwen2.5-3B-Instruct as the base model. We first describe the data preparation pipeline, then explore prompt strategies, fine-tune under both strategies (Baseline & AC-3) via SFT, and finally continue training the best per-prompt SFT checkpoint with GRPO under two reward seperate functions (bucketed & normalized), then compare both of them. Evaluation is performed after each stage.

2.1 Data Preparation

The ZebraLogic benchmark (Lin et al., 2025) provides 1,000 evaluation puzzles across four size categories — small, medium, large, and x-large — but does not release the generation source code. To obtain in-distribution training data, we reimplement and release the puzzle generator (Appendix G).

Each puzzle is parameterized by N houses and M attributes, covering all integer configurations from 2×2 to 6×6 (25 sizes). Attribute values are drawn from a pool of 25 named categories (e.g., Name, Color, Nationality, Drink). Given a randomly sampled solution, the generator constructs a weighted candidate pool of clues using six primitive relation types. A saturated subset is passed to the Z3 SMT solver (de Moura and Bjørner, 2008), which verifies uniqueness; clues are greedily removed as long as the solution remains uniquely determined, yielding a minimal clue set.

For SFT, we generate 1,000 training and 150 validation puzzles. Reasoning traces are obtained by prompting DeepSeek-R1 (Guo et al., 2025) on each puzzle under both prompt styles (Baseline and AC-3), then retaining only puzzles solved correctly under both, yielding 937 training and 139 validation examples. Each trace is verified against the ground-truth solution before inclusion. Our dataset size is informed by the "less is more" hypothesis: Zhou et al. (2023) demonstrate that 1,000 carefully curated examples suffice for strong instruction following, and Ye et al. (2025) show that even 817 high-quality reasoning traces yield state-of-the-art mathematical reasoning — suggesting that quality and in-distribution coverage matter more than scale for domain-specific SFT. For GRPO, we generate a separate set of 1,000 puzzles containing only the puzzle text and ground-truth solution, without reasoning traces.

2.2 Prompt Strategies

We adopt the prompt provided by Lin et al. (2025) (Appendix A) as our baseline. Additionally, motivated by the suitability of the AC-3 arc-consistency algorithm (Mackworth, 1977) for CSP-style reasoning, we design a prompt that replicates AC-3's constraint propagation steps in natural language (Appendix B). While the baseline prompt elicits more free-form reasoning followed by a structured JSON solution, the AC-3 prompt enforces a more structured reasoning structure with less reliance on a fixed output schema.

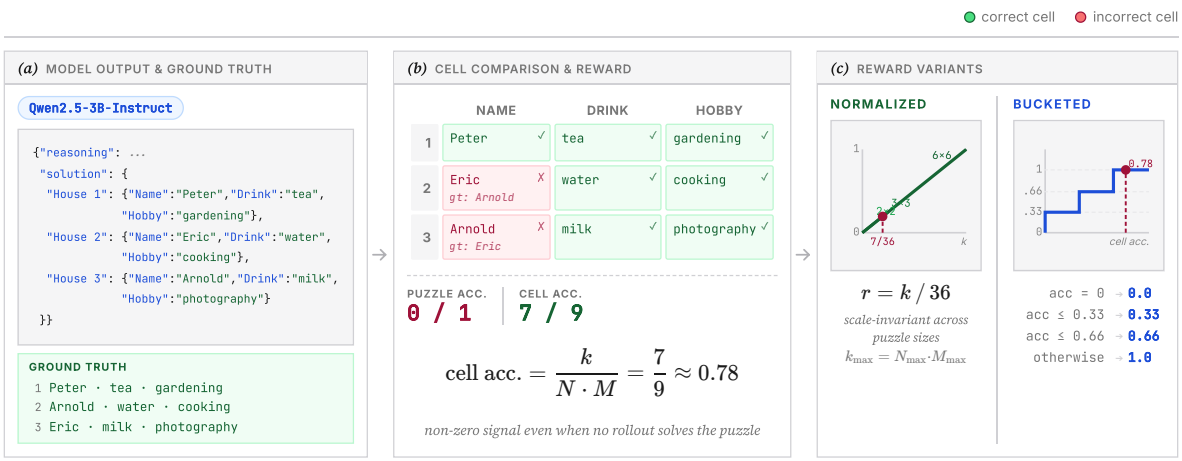


Figure 1: Cell-accuracy based rewarding. The model produces a structured JSON grid (a) for a 3×3 Zebra puzzle; comparing prediction to ground truth cell-by-cell (b) yields a dense reward signal: here two names are incorrect, so puzzle accuracy is 0, yet cell accuracy is $7/9$ and cell acc. $= k/(N \cdot M) \approx 0.78$. Panel (c) shows the two GRPO reward variants: normalized assigns $r = k/36$, making rewards scale-invariant across all puzzle sizes 2×2–6×6; bucketed maps cell accuracy to four reward buckets $\{0, 0.33, 0.66, 1.0\}$.

2.3 Supervised Fine-Tuning

We fine-tune Qwen2.5-3B-Instruct using LoRA (Hu et al., 2021) ($r = 64$, $\alpha = 128$, dropout $= 0.05$), with a learning rate of 2×10^{-4} under cosine decay (3% warmup), effective batch size of 16, and early stopping with patience 2 (max 8 epochs)(Appendix D). The model is trained independently under two prompt styles — Baseline and AC-3 (Appendix C) — yielding two fine-tuned configurations.

2.4 GRPO Post-Training

Following Tekin et al. (2025), we apply GRPO (Shao et al., 2024) — a memory-efficient variant of PPO (Schulman et al., 2017) — as a post-training stage over the best SFT checkpoint per prompt style. GRPO estimates advantages from within-group reward contrasts, eliminating the need for a separate critic model and making it well-suited for verifiable, rule-based rewards. We adopt the Cerebras hyperparameter configuration: learning rate 1×10^{-6} with linear decay and 20 warmup steps, $\beta = 0.005$, 16 rollouts per prompt, 2 gradient updates per generation step, temperature 0.65, and asymmetric DAPO-style clipping (Yu et al., 2025) ($\varepsilon = 0.2$, $\varepsilon_{\text{high}} = 0.28$).

We evaluate two reward functions (Figure 1). The *standard* variant computes cell accuracy and maps it to a discrete bucket: 0.0 for parse failure or no correct cell, 0.33 for accuracy $\leq 33\%$, 0.66 for $\leq 66\%$, and 1.0 otherwise. The *normalized* variant instead assigns a reward of $k/36$, where k is the number of correctly predicted cells and 36 is the maximum cell count in a 6×6 puzzle, ensuring that solving k cells yields the same reward regardless of puzzle scale.

2.5 Experimental Setup & Evaluation Metrics

All experiments were conducted on Google Colab using an NVIDIA A100 GPU (High RAM, 80 GB VRAM). The training environment was built on PyTorch, Hugging Face Transformers, Hugging Face Datasets, PEFT, TRL, Accelerate, bitsandbytes, vLLM, Z3 Solver, and Weights & Biases (wandb) was used for experiment tracking (Appendix H).

Following the evaluation protocol of Lin et al. (2025), we report *puzzle accuracy* (the percentage of puzzles for which the model produces a fully correct solution) and *cell accuracy* (the percentage of individual attribute-value assignments that are correctly predicted). In addition to these primary metrics, we track parse error counts to measure output format reliability, peak VRAM usage, and total runtime (Appendix E). For SFT runs, training and validation loss are recorded at the end of each epoch; for GRPO runs, reward-level metrics and parse error counts are continuously logged via wandb (Appendix F).

Models	Small puzzles (320 total)			Medium puzzles (280 total)		
	Puzzle acc. (%)	Cell acc. (%)	Parse err.	Puzzle acc. (%)	Cell acc. (%)	Parse err.
Base – Baseline prompt	15.62	50.96	6	0	26.91	47
Base – AC-3 prompt	18.75	53.06	8	0	22.60	1
Fine-tuned – Baseline prompt	57.81	80.22	**0**	3.57	47.07	4
Fine-tuned – AC-3 prompt	**66.56**	**83.47**	**0**	**9.64**	**52.88**	1
GRPO (bucketed) – Baseline prompt	54.69	81.80	**0**	4.29	49.40	**0**
GRPO (bucketed) – AC-3 prompt	42.50	71.62	1	6.43	50.06	3
GRPO (normalized) – Baseline prompt	60.94	83.16	**0**	6.43	50.42	2
GRPO (normalized) – AC-3 prompt	58.13	82.36	**0**	3.57	48.42	9

Table 1: Main results on ZEBRALOGIC test dataset for small and medium puzzle groups. Metrics are puzzle accuracy (exact-solve rate), cell accuracy (cell-wise accuracy), and parse errors (invalid/failed parses).

3 Results & Discussion

Results across all ZebraLogic categories show a consistent pattern: both prompt strategies benefit substantially from SFT, with performance dropping sharply beyond small puzzles and near-zero puzzle accuracy on large and extra-large instances. Cell accuracy remains non-zero at larger sizes, indicating partial reasoning persists where full solutions are not recovered.

A notable threshold effect emerges at medium size: the base model achieves zero puzzle accuracy under both prompt strategies, while SFT recovers nonzero puzzle accuracy (3.57% baseline, 9.64% AC-3), suggesting that fine-tuning instills enough structured reasoning to occasionally produce fully correct solutions at this scale, even if overall performance remains low.

SFT also substantially reduces parse errors across both prompts, with zero failures observed on small puzzles for all SFT runs. This improvement partially degrades under GRPO, and worsens with puzzle size (Appendix J) — the AC-3 prompt accumulates far more parse errors at large and x-large sizes than the baseline, consistent with its longer and more rigidly structured outputs straining the model's ability to maintain valid JSON formatting as context length grows.

The most striking finding concerns the interaction between prompt strategy and training stage. The AC-3 prompt outperforms the baseline under SFT, consistent with structured constraint propagation aiding trace-based learning. GRPO reverses this: the baseline prompt sees modest gains while the AC-3 prompt degrades substantially. The AC-3 prompt's rigid structure incurs longer generation times and higher parse error rates — properties that suit imitation learning but work against GRPO's rollout-based exploration. The prompt that best supports SFT is not necessarily the one that best supports RL post-training.

To probe whether in-distribution gains reflect genuine reasoning improvements or format-specific overfitting, we evaluate all configurations on Enigmata-Eval (Appendix I). The GRPO bucketed baseline achieves the strongest out-of-domain accuracy (8.00%), while AC-3 GRPO variants collapse to zero, mirroring the in-distribution pattern. Overall out-of-domain performance remains low, suggesting that gains are at least partially puzzle-format-specific rather than reflecting broad reasoning transfer.

Overall, SFT accounts for the overwhelming majority of gains over zero-shot, with GRPO contributing only marginally.

4 Conclusion & Future Work

We present a reproducible end-to-end training pipeline for Zebra puzzle solving and for testing the logical reasoning capabilities of LLMs, and we reimplement & release the ZEBRALOGIC generator to enable further investigation. Our results show that fine-tuning Qwen2.5-3B-Instruct on approximately 937 in-distribution examples yields substantial improvements for both of prompt strategies, including nonzero puzzle accuracy on medium puzzles where the base model fails entirely. Finally, the unexpected interaction between prompt structure and GRPO post-training warrants deeper study; our released pipeline provides the infrastructure to do so systematically.

Limitations

This work evaluates a single model, Qwen2.5-3B-Instruct, trained under the compute constraints of

a single NVIDIA A100 GPU on Google Colab. Generalization across model families, scales, and training budgets remains an open question.

Ethics Statement

We used AI assistance (Claude 4.6 Sonnet) for drafting and polishing the manuscript. All scientific claims, analyzes, and conclusions were reviewed and verified by the authors.

References

Rikhil Amonkar, Ceyhun Efe Kayan, Qimei Lai, Ronan Le Bras, and Li Zhang. 2026. A reality check of language models as formalizers on constraint satisfaction problems. *Preprint*, arXiv:2505.13252.

Alexander Beiser, David Penz, and Nysret Musliu. 2025. Intermediate languages matter: Formal choice drives neurosymbolic LLM reasoning. *Preprint*, arXiv:2502.17216.

Jiangjie Chen, Qianyu He, Siyu Yuan, Aili Chen, Zhicheng Cai, Weinan Dai, Hongli Yu, Qiying Yu, Xuefeng Li, Jiaze Chen, Hao Zhou, and Mingxuan Wang. 2025. Enigmata: Scaling logical reasoning in large language models with synthetic verifiable puzzles. *CoRR*, abs/2505.19914.

Leonardo Mendonça de Moura and Nikolaj S. Bjørner. 2008. Z3: an efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems, 14th International Conference, TACAS 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29-April 6, 2008. Proceedings*, Lecture Notes in Computer Science, pages 337–340. Springer.

Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Sean Welleck, Peter West, Chandra Bhagavatula, Ronan Le Bras, Jena D. Hwang, Soumya Sanyal, Xiang Ren, Allyson Ettinger, Zaïd Harchaoui, and Yejin Choi. 2023. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, and 1 others. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638.

Prakhar Gupta and Vaibhav Gupta. 2025. Bootstrapped mixed rewards for RL post-training: Injecting canonical action order. *CoRR*, abs/2512.04277.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. 2021. LoRA: Low-rank adaptation of large language models. *CoRR*, abs/2106.09685.

Bill Yuchen Lin, Ronan Le Bras, Kyle Richardson, Ashish Sabharwal, Radha Poovendran, Peter Clark, and Yejin Choi. 2025. ZebraLogic: On the scaling limits of LLMs for logical reasoning. In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, Proceedings of Machine Learning Research. PMLR.

Alan K. Mackworth. 1977. Consistency in networks of relations. *Artificial Intelligence*, 8(1):99–118.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *CoRR*, abs/2402.03300.

Engin Tekin, Anisha Garg, and Ganesh Venkatesh. 2025. From zero to Sudoku hero: An RL adventure. Cerebras Blog.

Nemika Tyagi, Mihir Parmar, Mohith Kulkarni, Aswin Rrv, Nisarg Patel, Mutsumi Nakamura, Arindam Mitra, and Chitta Baral. 2024. Step-by-step reasoning to solve grid puzzles: Where do LLMs falter? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 19898–19915, Miami, Florida, USA. Association for Computational Linguistics.

Zhen Hao Wong, Jingwen Deng, Runming He, Zirong Chen, Qijie You, Hejun Dong, Hao Liang, Chengyu Shen, Bin Cui, and Wentao Zhang. 2025. Logicpuzzlerl: Cultivating robust mathematical reasoning in llms via reinforcement learning. *CoRR*, abs/2506.04821.

Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. 2025. LIMO: Less is more for reasoning. In *Conference on Language Modeling, COLM 2025*.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, and 16 others. 2025. DAPO: An open-source LLM reinforcement learning system at scale. *Preprint*, arXiv:2503.14476.

Chunting Zhou, Pengfei Liu, Puxin Xu, Srinivasan Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, Susan Zhang, Gargi Ghosh, Mike Lewis, Luke Zettlemoyer, and Omer Levy. 2023. LIMA: Less is more for alignment. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023,*

NeurIPS 2023, New Orleans, LA, USA, December 10-16, 2023.

A Baseline Prompt

```
# Example Puzzle

There are 3 houses, numbered 1 to 3 from left to
     right, as seen from
across the street. Each house is occupied by a
     different person. Each
house has a unique attribute for each of the
     following characteristics:
 - Each person has a unique name: `Peter`, `Eric`, `
     Arnold`.
 - Each person has a unique favorite drink: `tea`, `
     water`, `milk`

## Clues for the Example Puzzle

1. Peter is in the second house.
2. Arnold is directly left of the one who only
     drinks water.
3. The one who only drinks water is directly left of
      the person
   who likes milk.

## Answer to the Example Puzzle

{
    "reasoning": "Given Clue 1, we know Peter is in
     House 2.
    According to Clue 2, Arnold is directly left of
     the one who only
    drinks water. The person in House 3 cannot be on
      the left of
    anyone, so Arnold must be in House 1. Thus,
     Peter drinks water,
    and Eric lives in House 3. Then, according to
     Clue 3, Eric drinks
    milk. Therefore, Arnold drinks tea.",
    "solution": {
        "House 1": {"Name": "Arnold", "Drink": "tea
     "},
        "House 2": {"Name": "Peter",  "Drink": "
     water"},
        "House 3": {"Name": "Eric",   "Drink": "milk
     "}
    }
}

# Puzzle to Solve

{puzzle}

# Instruction

Now please solve the above puzzle. Present your
     reasoning and solution
in the following json format:

{json_template}
```

B AC-3 Simplified Prompt

```
# Example Puzzle

There are 3 houses, numbered 1 to 3 from left to
     right, as seen from
across the street. Each house is occupied by a
     different person. Each
house has a unique attribute for each of the
     following characteristics:
 - Each person has a unique name: `Peter`, `Eric`, `
     Arnold`.
 - Each person has a unique favorite drink: `tea`, `
     water`, `milk`

## Clues for the Example Puzzle

1. Peter is in the second house.
2. Arnold is directly left of the one who only
     drinks water.
3. The one who only drinks water is directly left of
      the person
   who likes milk.

## Answer to the Example Puzzle

### Reasoning
Goal: Solve placement for Houses [1, 2, 3]

STEP 0 - Initialize Domains

All-different constraints apply.

House1.Name = {Peter, Eric, Arnold}
House2.Name = {Peter, Eric, Arnold}
House3.Name = {Peter, Eric, Arnold}

House1.Drink = {tea, water, milk}
House2.Drink = {tea, water, milk}
House3.Drink = {tea, water, milk}

STEP 1 - Apply Clue 1 (Peter is in the second house)

House2.Name = {Peter}

Any value unsupported in remaining domains is
     removed:
House1.Name = {Eric, Arnold}
House3.Name = {Eric, Arnold}

STEP 2 - Apply Clue 3 (Water is directly left of
     Milk)

Valid (Water, Milk) pairs: (1,2), (2,3)

Any value with no valid pair partner is removed:
Water in {1,2}
Milk  in {2,3}

STEP 3 - Apply Clue 2 (Arnold directly left of Water
     )

Check each remaining value for a valid partner in
     the other domain:
Water=1 -> Arnold=0 -> impossible, remove

Therefore:
Arnold = 1, Water = 2, Milk = 3 (forced from pairing
     )

STEP 4 - Final Propagation (Uniqueness)

House1.Name = Arnold, House2.Name = Peter, House3.
     Name = Eric
House2.Drink = water, House3.Drink = milk, House1.
     Drink = tea

### Solution
{
    "House 1": {"Name": "Arnold", "Drink": "tea"},
    "House 2": {"Name": "Peter",  "Drink": "water"},
    "House 3": {"Name": "Eric",   "Drink": "milk"}
}

# Puzzle to Solve

{puzzle}

# Instruction

Solve the above puzzle. At each step apply the clue,
      then remove any
value that has no valid partner in the connected
     domain. Collapse and
propagate when a domain reaches one value.
Do not use trial-and-error or backtracking unless no
      deterministic
inference is possible.
Present your reasoning and solution in the following
      json format:

{json_template}
```

C Example: Base Model with SFT (AC-3 Prompt)

Table 2 shows model outputs on a small (2×2) puzzle under the AC-3 prompt. The SFT model

produces a structured, step-by-step derivation that correctly propagates constraints and arrives at the ground-truth solution.

Table 2: Result comparison on a 2×2 puzzle (AC-3 prompt). The SFT model solves the puzzle correctly.

Puzzle	There are 2 houses. Names: Eric, Arnold. Pets: dog, cat. Clues: (1) Eric is somewhere to the left of Arnold. (2) The person who owned a dog is not in the first house.
SFT output	*reasoning:* "From clue 1, Eric is somewhere to the left of Arnold. With only two houses, this forces Eric to be in house 1 and Arnold in house 2. From clue 2, the person who owns a dog is not in the first house, so the dog owner must be in house 2. Since Arnold is in house 2, he owns the dog. Therefore, Eric must own the cat. All clues are satisfied." *solution:* House 1: Eric, cat House 2: Arnold, dog
Ground truth	House 1: Eric, cat House 2: Arnold, dog
Puzzle acc.	1/1 Cell acc.: 4/4

D SFT Loss Curves

Tables 3 and 4 report per-epoch training and validation loss for SFT under the baseline and AC-3 prompts, respectively. Early stopping triggered after epoch 5 (patience 2); "—" indicates epochs not run.

Table 3: SFT loss per epoch (Baseline prompt). Format: train / val.

	Ep 1	**Ep 2**	**Ep 3**	**Ep 4**	**Ep 5**	**Ep 6**	**Ep 7**	**Ep 8**
Train	0.9082	0.0823	0.0672	0.0536	0.0399	—	—	—
Val	0.0917	0.0835	0.0824	0.0855	0.0975	—	—	—

Baseline initial loss before training (epoch 0): 12.1594.

Table 4: SFT loss per epoch (AC-3 prompt). Format: train / val.

	Ep 1	**Ep 2**	**Ep 3**	**Ep 4**	**Ep 5**	**Ep 6**	**Ep 7**	**Ep 8**
Train	0.7730	0.1578	0.1304	0.1056	0.0805	—	—	—
Val	0.1886	0.1720	0.1696	0.1748	0.1994	—	—	—

AC-3 initial loss before training (epoch 0): 8.0284.

E Runtime and VRAM Usage

Table 5: Runtime and peak VRAM usage per experimental stage within the Zebra Logic small test dataset. All runs on a single NVIDIA A100 80 GB (Google Colab High RAM).

Stage	Config	Runtime	Peak VRAM
Base model	Baseline prompt	36 m	15.7 GB
Base model	AC-3 prompt	24 m	16.0 GB
SFT	Baseline prompt	7.52 m	11.7 GB
SFT	AC-3 prompt	19.51 m	13.4 GB
GRPO	Baseline prompt (bucketed reward)	7.23 m	9.7 GB
GRPO	Baseline prompt (normalized reward)	7.18 m	9.7 GB
GRPO	AC-3 prompt (bucketed reward)	14.46 m	9.6 GB
GRPO	AC-3 prompt (normalized reward)	15.11 m	9.6 GB

F GRPO Reward Curves

Figures 2 and 3 summarize GRPO post-training dynamics as logged via Weights & Biases.

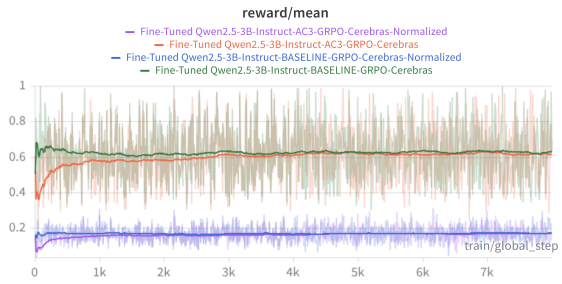


Figure 2: GRPO post-training: mean group reward over training steps.

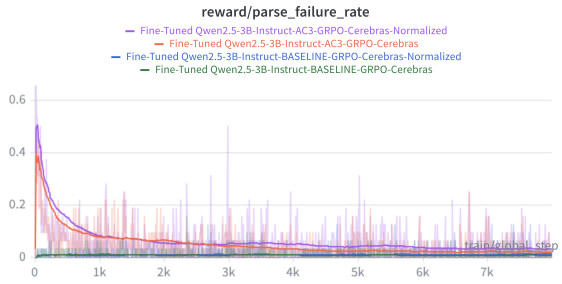


Figure 3: GRPO post-training: JSON parse failure rate over training steps.

G Dataset Generation Algorithm

The following pseudocode describes the full pipeline for generating uniquely-solvable ZebraLogic constraint puzzles. The pipeline has four stages: solution generation, clue candidate generation, Z3-based uniqueness verification, and greedy clue minimisation.

Data Definitions

Attribute domains ($\mathcal{A}_{\text{all}}$ and $\mathcal{V}_a$). The system defines 25 attributes; each has exactly 6 possible values. For any puzzle of size $N \times M$, M attributes are sampled and their value pools are sliced to N items.

Table 6: Attribute pool: 25 attributes, each with 6 possible values.

Attribute	Sample values
Name	Alice, Peter, Carol, Bob, Eric, Arnold
Color	red, purple, blue, green, white, yellow
Nationality	german, norwegian, dane, swede, brit, chinese
Animal	dog, rabbit, horse, cat, fish, bird
Drink	coffee, milk, tea, root beer, boba tea, water
Cigar	yellow monster, blue master, blends, dunhill, prince, pall mall
Food	soup, stir fry, grilled cheese, spaghetti, pizza, stew
Flower	carnations, roses, iris, tulips, lilies, daffodils
PhoneModel	google pixel 6, xiaomi mi 11, samsung galaxy s21, ...
Children	Fred, Alice, Bella, Timothy, Meredith, Samantha
Smoothie	desert, dragonfruit, cherry, lime, watermelon, blueberry
Birthday	april, sept, mar, may, jan, feb
Occupation	artist, nurse, teacher, lawyer, engineer, doctor
Height	very short, super tall, tall, short, very tall, average
CarModel	toyota camry, tesla model 3, honda civic, bmw 3 series, ...
FavoriteSport	baseball, soccer, tennis, volleyball, basketball, swimming
MusicGenre	hip hop, classical, country, pop, rock, jazz
BookGenre	romance, fantasy, science fiction, biography, mystery, ...
HairColor	gray, brown, blonde, auburn, black, red
Mother	Penny, Holly, Aniya, Kailyn, Janelle, Sarah
HouseStyle	craftsman, ranch, modern, victorian, colonial, mediterranean
Education	master, trade school, high school, bachelor, doctorate, associate
Hobby	cooking, painting, photography, gardening, knitting, woodworking
Vacation	beach, city, cultural, cruise, mountain, camping
Pet	bird, hamster, fish, rabbit, dog, cat

Puzzle sizes and size ranking. A puzzle is parameterised by N (houses) $\times$ M (attributes). All 25 supported sizes are ranked by sizes and grouped into four tiers used by the weighted distribution strategy.

Tier	Sizes ($N \times M$)	Ranks	Allocation
Small	$2\times2,\ 2\times3,\ 2\times4,\ 2\times5,\ 2\times6,\ 3\times2,\ 3\times3,\ 4\times2$	1–8	10%
Medium	$3\times4,\ 3\times5,\ 4\times3,\ 4\times4,\ 5\times2,\ 5\times3$	9–14	20%
Large	$3\times6,\ 4\times5,\ 5\times4,\ 6\times2,\ 6\times3$	15–19	30%
X-Large	$4\times6,\ 5\times5,\ 5\times6,\ 6\times4,\ 6\times5,\ 6\times6$	20–25	40%

Algorithms

Algorithm 1 SolvePuzzleGeneration

Require: Number of houses N, number of attributes M
Ensure: Minimally-clued puzzle object, or None

$pg \leftarrow$ INITPUZZLE(N, M)
GENERATESOLUTION(pg)
$C \leftarrow$ GENERATEALLPOSSIBLECLUES(pg)
$k \leftarrow \min(|C|, 80)$; $C \leftarrow C[0 \,..\, k{-}1]$
Verify C with Z3CHECKER(N, pg, C); **if** not unique **return** None
$R \leftarrow \emptyset$
for $i \leftarrow 0$ **to** $|C| - 1$ **do**
    $C' \leftarrow [c_j \mid j \neq i \ \wedge \ j \notin R]$
    if Z3CHECKER(N, pg, C') is unique **then**
        $R \leftarrow R \cup \{i\}$
    end if
end for
pg.clues $\leftarrow [c_i \mid i \notin R]$
return pg

Algorithm 2 GenerateAllPossibleClues

Require: Puzzle pg (solution known)
Ensure: Weighted, deduplicated clue list C_{pool}, $|C_{\text{pool}}| \leq 500$

raw $\leftarrow [\,]$
Phase 1 — Positional (per attribute, per value):
for each $a \in$ *pg.selected_attrs*, each $v \in$ *pg.data*$[a]$ **do**
    $p \leftarrow$ GETPOS(pg, a, v)
    Append FOUNDAT(a, v, p) [weight 3.5]
    Sample $p' \in \{1, \ldots, N\} \setminus \{p\}$; Append NOTAT$(a, v, p')$ [weight 1.0]
end for
Phase 2 — Relational (per attribute pair):
for each $(a_1, a_2) \in \binom{\textit{pg.selected_attrs}}{2}$ **do**
    for each house row $h \in$ *pg.solution* **do**
        Let $v_1 = h[a_1]$, $v_2 = h[a_2]$
        Append SAMEHOUSE(a_1, v_1, a_2, v_2) [weight 5.0]
        Append NOTSAME$(a_1, v_1, a_2, \text{sample}(W))$ [weight 1.0]
    end for
    for each $v_1 \in$ *pg.data*$[a_1]$, each $v_2 \in$ *pg.data*$[a_2]$ **do**
        $d \leftarrow$ GETPOS(pg, a_2, v_2) $-$ GETPOS(pg, a_1, v_1)
        if $d = 1$ **then** Append DIRECTLEFT [weight 4.5]
        end if
        if $d = -1$ **then** Append DIRECTRIGHT [weight 4.5]
        end if
        if $|d| = 1$ **then** Append SIDEBYSIDE [weight 4.5]
        end if
        if $d > 0$ **then** Append LEFT [weight 2.5]
        end if
        if $d < 0$ **then** Append RIGHT [weight 2.5]
        end if
        if $|d| = 2$ **then** Append ONEBETWEEN [weight 1.5]
        end if
        if $|d| = 3$ **then** Append TWOBETWEEN [weight 1.5]
        end if
    end for
end for
sampled $\leftarrow$ draw $\min(|\textit{raw}|, 500)$ items proportional to weights
Remove duplicates $\Rightarrow C_{\text{pool}}$; **return** C_{pool}

Algorithm 3 Z3Checker

Require: N, puzzle pg, clue list C
Ensure: Boolean: is C uniquely satisfiable?

Create SMT solver Σ
for each a, each v **do** Declare $x_{a,v} \in \mathbb{Z}$ with $1 \le x_{a,v} \le N$
end for
for each a **do** Add DISTINCT$(\{x_{a,v}\})$ to Σ
end for
for each clue $c \in C$ **do** Translate c to SMT constraint and add to Σ
end for
if Σ is UNSAT **then return** *False*
end if
$\mathcal{M} \leftarrow \Sigma$.MODEL()
Σ.PUSH(); Add $\bigvee_{x \in \mathcal{M}} (x \neq \mathcal{M}[x])$ to Σ
unique $\leftarrow$ (Σ is UNSAT); Σ.POP()
return *unique*

Table 7: SMT constraint encoding for each clue type.

Clue type	SMT constraint
FOUNDAT(a, v, p)	$x_{a,v} = p$
NOTAT(a, v, p)	$x_{a,v} \neq p$
SAMEHOUSE(a_1, v_1, a_2, v_2)	$x_{a_1,v_1} = x_{a_2,v_2}$
NOTSAME(a_1, v_1, a_2, v_2)	$x_{a_1,v_1} \neq x_{a_2,v_2}$
DIRECTLEFT(a_1, v_1, a_2, v_2)	$x_{a_1,v_1} = x_{a_2,v_2} - 1$
DIRECTRIGHT(a_1, v_1, a_2, v_2)	$x_{a_1,v_1} = x_{a_2,v_2} + 1$
SIDEBYSIDE(a_1, v_1, a_2, v_2)	$\lvert x_{a_1,v_1} - x_{a_2,v_2} \rvert = 1$
LEFT(a_1, v_1, a_2, v_2)	$x_{a_1,v_1} < x_{a_2,v_2}$
RIGHT(a_1, v_1, a_2, v_2)	$x_{a_1,v_1} > x_{a_2,v_2}$
ONEBETWEEN(a_1, v_1, a_2, v_2)	$\lvert x_{a_1,v_1} - x_{a_2,v_2} \rvert = 2$
TWOBETWEEN(a_1, v_1, a_2, v_2)	$\lvert x_{a_1,v_1} - x_{a_2,v_2} \rvert = 3$

H Library Versions

Table 8: Python library versions used in all experiments.

Library	**Version**
torch	2.10.0+cu128
transformers	5.0.0
peft	0.18.1
accelerate	1.13.0
bitsandbytes	0.49.2
datasets	4.0.0
tqdm	4.67.3
safetensors	0.7.0
torchao	0.17

I Enigmata Out-of-Domain Results

The table below reports accuracy on the Enigmata-Eval benchmark across all training configurations within their specified Zebra Logic task. Each difficulty split contains 50 puzzles (150 total). Results reflect out-of-domain generalisation: all models were trained exclusively on Zebra puzzles and received no Enigmata-specific supervision.

Table 9: Enigmata-Eval accuracy by model configuration and size. All models trained on Zebra puzzles only (out-of-domain evaluation).

Model / Config	**Easy (%)**	**Medium (%)**	**Hard (%)**	**Overall (%)**
Base model	4.00	0.00	0.00	1.33
Fine-tuned – Baseline prompt	16.00	**4.00**	0.00	6.67
Fine-tuned – AC-3 prompt	2.00	0.00	0.00	0.67
GRPO (bucketed) – Baseline prompt	**20.00**	**4.00**	0.00	**8.00**
GRPO (bucketed) – AC-3 prompt	0.00	0.00	0.00	0.00
GRPO (normalized) – Baseline prompt	10.00	0.00	0.00	3.33
GRPO (normalized) – AC-3 prompt	0.00	0.00	0.00	0.00

J Extended ZebraLogic Results (All Size Categories)

The table below extends the main results (Table 1) with all four size categories.

Models	Small (320 total)			Medium (280 total)			Large (240 total)			X-Large (160 total)		
	Puz. (%)	Cell (%)	Err.	Puz. (%)	Cell (%)	Err.	Puz. (%)	Cell (%)	Err.	Puz. (%)	Cell (%)	Err.
Fine-tuned – Baseline prompt	57.81	80.22	**0**	3.57	47.07	4	0	35.32	10	0	22.81	38
Fine-tuned – AC-3 prompt	**66.56**	**83.47**	**0**	**9.64**	**52.88**	1	**0.50**	29.61	50	0	14.28	101
GRPO (bucketed) – Baseline prompt	54.69	81.80	**0**	4.29	49.40	**0**	0	36.38	**2**	0	**25.86**	**6**
GRPO (bucketed) – AC-3 prompt	42.50	71.62	1	6.43	50.06	3	0	35.59	**2**	0	23.79	21
GRPO (normalized) – Baseline prompt	60.94	83.16	**0**	6.43	50.42	2	**0.50**	**37.89**	**2**	0	25.20	22
GRPO (normalized) – AC-3 prompt	58.13	82.36	**0**	3.57	48.42	9	**0.50**	36.71	4	0	25.28	26

Table 10: Full results on Zebra Logic test dataset.